

DAY -17

JAVA OOP (Object-Oriented Programming)

1. What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that uses **classes** and **objects** to design programs.

It combines **data (variables)** and **behavior (methods)** into a single unit called an object.

Simple Definition

OOP is a programming method that represents real-world objects in software.

Real-Life Example

Car

- Data → Color, Speed, Model
- Methods → Start(), Stop(), Drive()

Advantages of OOP

- Code Reusability
 - Security
 - Easy Maintenance
 - Easy to Understand
 - Better Organization
-

2. Class

A **class** is a blueprint or template used to create objects.

Example

```
class Car {  
    String color;  
    int speed;
```

```
void drive() {  
    System.out.println("Car is driving");  
}  
}
```

Simple Definition

A class is a blueprint for creating objects.

3. Object

An **object** is a real instance of a class.

Example

```
Car c1 = new Car();
```

Access Members

```
c1.color = "Red";
```

```
c1.speed = 100;
```

```
c1.drive();
```

Output

Car is driving

Real-Life Example

Class → Student

Objects:

- Rahul
 - Sindhu
 - Ravi
-

Difference Between Class and Object

Class	Object
Blueprint	Real instance

Class	Object
--------------	---------------

No memory	Uses memory
-----------	-------------

Created once	Many objects can be created
--------------	-----------------------------

4. The Four Pillars of OOP

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

5. Encapsulation

Encapsulation is the process of wrapping data and methods into one class and protecting data using private variables.

Simple Definition

Encapsulation means hiding data and accessing it through public methods.

Example

```
class BankAccount {  
    private double balance;  
    public void deposit(double amount) {  
        balance += amount;  
    }  
    public double getBalance() {  
        return balance;  
    }  
}
```

Object

```
BankAccount acc = new BankAccount();
```

```
acc.deposit(5000);  
System.out.println(acc.getBalance());
```

Output

5000

Real-Life Example

ATM Machine

You cannot change your account balance directly.

Advantages

- Data Security
 - Data Hiding
 - Controlled Access
 - Easy Maintenance
-

6. Inheritance

Inheritance is the process where one class acquires the properties and methods of another class.

Keyword

extends

Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
}
```

Object

```
Dog d = new Dog();
```

```
d.eat();
```

```
d.bark();
```

Output

Animal eats

Dog barks

Real-Life Example

Animal

↓

Dog

Advantages

- Code Reuse
- Less Coding
- Easy Maintenance

7. Polymorphism

Polymorphism means one method can perform different tasks.

Meaning

- Poly = Many
- Morph = Forms

One Name → Many Forms

Types

1. Method Overloading
2. Method Overriding

8. Method Overloading

Definition

Same method name with different parameters.

Compile-Time Polymorphism.

Example

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Output

5

6.0

9

Real-Life Example

Camera

Photo()

Photo(filter)

Photo(filter, timer)

9. Method Overriding

Method Overriding means a child class provides a new implementation of the parent class method.

Runtime Polymorphism.

Example

```
class Shape {  
    void draw() {  
        System.out.println("Drawing Shape");  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

Object

```
Shape s = new Circle();  
s.draw();
```

Output

Drawing Circle

Real-Life Example

Animal → sound()

Dog → Bow Bow

Cat → Meow

Difference Between Overloading and Overriding

Overloading	Overriding
Same class	Parent & Child class
Different parameters	Same parameters
Compile Time	Runtime

Overloading

Overriding

Inheritance not required Inheritance required

10. Abstraction

Abstraction is the process of hiding implementation details and showing only essential features.

Simple Definition

Hide complexity and show only important features.

Achieved Using

- Abstract Class
- Interface

Example

```
abstract class Vehicle {  
    abstract void start();  
    void stop() {  
        System.out.println("Vehicle stopped");  
    }  
}  
  
class Bike extends Vehicle {  
    void start() {  
        System.out.println("Bike started");  
    }  
}
```

Object

```
Vehicle v = new Bike();  
v.start();  
v.stop();
```

Output

Bike started

Vehicle stopped

Real-Life Example

ATM Machine

You know how to withdraw money but not how the ATM works internally.

Advantages

- Hides Complexity
 - Better Security
 - Easy Maintenance
-

Difference Between Encapsulation and Abstraction

Encapsulation

Hides data

Provides data security

Uses private variables and getters/setters

Focuses on protecting data

Abstraction

Hides implementation

Reduces complexity

Uses abstract class and interface

Focuses on showing only essential features

Real-Life Example

Encapsulation → Bank Account

Abstraction → ATM Machine

11. Constructor

A constructor is a special method that is called automatically when an object is created.

Rules

- Same name as class
- No return type
- Executes automatically
- Initializes object values

Types

1. Default Constructor
2. Parameterized Constructor

Example

```
class Student {  
    String name;  
  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Object

```
Student s = new Student("Sindhu", 21);
```

Real-Life Example

Buying a mobile phone with predefined RAM, storage, and color.

12. this Keyword

this refers to the current object.

Uses

- Access current object variables
- Call current constructor
- Remove variable ambiguity

Example

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

```
}  
  
}
```

Real-Life Example

"This is my book."

13. super Keyword

super refers to the parent class.

Uses

- Call parent constructor
- Access parent variables
- Call parent methods

Example

```
class Person {  
    Person() {  
        System.out.println("Person Constructor");  
    }  
}  
  
class Student extends Person {  
    Student() {  
        super();  
        System.out.println("Student Constructor");  
    }  
}
```

Output

Person Constructor

Student Constructor

Real-Life Example

A child asking help from a parent.

14. Tightly Coupled (Strictly Coupled)

Definition

Classes are directly dependent on each other.

Example

```
class Engine {  
    void start() {  
        System.out.println("Engine Started");  
    }  
}  
  
class Car {  
    Engine engine = new Engine();  
    void drive() {  
        engine.start();  
    }  
}
```

Advantages

- Simple to write

Disadvantages

- Difficult to modify
- Difficult to test
- Less reusable

Real-Life Example

A TV that works only with one specific remote.

15. Loosely Coupled

Classes have less dependency and can work independently.

Example

```
interface Engine {  
    void start();  
}  
  
class PetrolEngine implements Engine {  
    public void start() {  
        System.out.println("Petrol Engine Started");  
    }  
}  
  
class Car {  
    Engine engine;  
    Car(Engine engine) {  
        this.engine = engine;  
    }  
    void drive() {  
        engine.start();  
    }  
}
```

Advantages

- Flexible
- Easy to modify
- Easy to test
- Highly reusable

Real-Life Example

A TV that works with any compatible universal remote.

Difference Between Tightly Coupled and Loosely Coupled

Tightly Coupled Loosely Coupled

High dependency Low dependency

Hard to modify Easy to modify

Less flexible More flexible

Less reusable More reusable

-
- **OOP:** Programming using classes and objects.
 - **Class:** A blueprint for creating objects.
 - **Object:** A real instance of a class.
 - **Encapsulation:** Wrapping data and methods together and protecting data.
 - **Inheritance:** One class acquires properties and methods of another class.
 - **Polymorphism:** One method name with many forms.
 - **Method Overloading:** Same method name with different parameters.
 - **Method Overriding:** Child class changes the parent class method.
 - **Abstraction:** Hiding implementation details and showing only essential features.
 - **Constructor:** A special method that initializes an object automatically.
 - **this:** Refers to the current object.
 - **super:** Refers to the parent class.
 - **Tightly Coupled:** Classes are strongly dependent on each other.
 - **Loosely Coupled:** Classes have less dependency and are more flexible.